Московский авиационный институт (национальный исследовательский университет)

Факультет информационных технологий и прикладной математики

Кафедра вычислительной математики и программирования

Лабораторная работа №2 по курсу «Информационный поиск»

Студент: П. А. Земсков
Преподаватель: А. А. Кухтичев
Группа: М8О-401Б-23

Дата: 10.10.2026
Оценка:
Подпись:

Москва, 2026

Лабораторная работа №2 «Поисковый робот»

Необходимо написать поисковый робот — компонент обкачки документов, используя любой язык программирования.

- Единственным аргументом поисковому роботу подаётся путь до yaml-конфига, содержащий:
  - данные для базы данных в секции `db`;
  - данные для робота в секции `logic`: задержка между обкачкой страницы;
  - любые другие данные, необходимые для реализации логики поискового робота.
- Сохранять в базе данных (например, MongoDB) документы со следующими полями:
  - url, нормализованный;
  - «сырой» html-текст документа;
  - название источника;
  - дата обкачки документа в формате Unix time stamp.
- Поисковый робот можно остановить в любой момент, и при повторном запуске робот должен начать с того документа, с которого он остановился.
- Периодически он должен уметь переобкачивать документы, которые уже есть в базе, но только в том случае, если они изменились.

1 Описание

1 Источник данных

Робот обкачивает русские Викиновости (`ru.wikinews.org`) — новостной проект на движке MediaWiki, того же, что и Википедия из первой лабораторной работы. В отличие от первой работы, статьи здесь не выбираются случайно через API, а обходятся по ссылкам: робот загружает обычные HTML-страницы `/wiki/<название>` в том виде, в каком их получает браузер («сырой» HTML со всей вёрсткой). Авторизация не нужна. Запросы отправляются с идентифицирующим `User-Agent` (с адресом для связи), как требуют правила Викимедиа, файл `robots.txt` сайта соблюдается.

2 Общее устройство

Робот — программа `main.py` на Python 3 (библиотеки `requests`, `BeautifulSoup`, `pymongo`, `PyYAML`); вся логика — один класс `Crawler`. Единственный аргумент программы — путь до YAML-конфига. Хранилище — MongoDB, две коллекции: `documents` (обкачанные документы) и `documents_queue` (очередь URL).

Алгоритм — обход в ширину:

1. в очередь добавляются стартовые URL (главная страница);
2. из очереди берётся самый старый необработанный URL, страница загружается;
3. документ сохраняется в `documents`;
4. из тела статьи извлекаются ссылки; подходящие URL, которых ещё нет в очереди, добавляются в неё;
5. URL помечается обработанным, выдерживается пауза, шаг 2 повторяется, пока очередь не опустеет или не будет достигнут лимит документов.

3 Конфигурация

В конфиге три обязательные секции: `db` (подключение к MongoDB), `logic` (параметры робота) и `source` (источник: название, домен, стартовые URL, закрытые пространства имён); необязательная секция `logging`.

```
db:
  host: localhost
  port: 27017
  database: wikinews
  collection: documents # queue lives in "<collection>_queue"
```

```
6
7  logic:
8    user_agent: "info-search-lab2/1.0 (student crawler; <e-mail>)"
9    delay_between_requests: 1.0 # seconds between requests
10   max_documents: 30000 # 0 = no limit
11   refresh_interval_days: 7 # re-check documents older than this
12   retry_attempts: 3 # HTTP-level retries and attempts per URL
13   timeout_seconds: 30
14
15 source:
16   name: Wikinews
17   domain: ru.wikinews.org
18   seeds: [<main page URL>]
19   skip_namespaces: [<Category, Special, Talk, User, File, ...>]
20
21 logging: {level: INFO, file: crawler.log}
```

4 Схема данных в MongoDB

Коллекция `documents`, один документ на страницу:

Поле	Описание
`url`	нормализованный канонический URL; уникальный индекс
`html_content`	«сырой» HTML страницы
`source`	название источника (`Wikinews`)
`crawl_timestamp`	дата обкачки, Unix time stamp
`content_hash`	MD5 текста статьи (для проверки изменений)
`title`, `content_length`, `text_length`	заголовок, размер сырого HTML в байтах, длина текста статьи в символах
`status_code`, `last_modified`	код ответа и заголовок `Last-Modified` (нужен для условного запроса при переобкачке)
`history`	предыдущие версии изменившихся документов (хэш и дата)

Коллекция `documents_queue`, одна запись на URL: `_id` — URL, `state` (`new` / `done` / `failed` / `error`), `tries` (число неудачных попыток), `refresh` (URL взят на переобкачку), `added` и `done_at` (время постановки в очередь и обработки).

5 Очередь URL и защита от повторных загрузок

- **Нормализация.** Схема приводится к `https`, хост — к нижнему регистру, отбрасываются фрагмент (`#...`) и query, пробелы заменяются на `_`, процентное кодирование приводится к единому виду. Ссылки `/wiki/A%20B` и `/wiki/A_B` —

один и тот же URL.

- **Очередь хранится в MongoDB**, `_id` записи — сам URL, добавление идёт через `upsert` с `$setOnInsert`. Поэтому URL попадает в очередь один раз за всё время жизни базы, а не только за один запуск. Для документов дополнительно создан уникальный индекс по `url`.

- **Фильтр ссылок.** Берутся только ссылки из тела статьи (`#mw-content-text`; меню и подвал сайта не обходятся) на домен `ru.wikinews.org` с путём `/wiki/`. Служебные пространства имён (Категория, Служебная, Обсуждение, Участник, Файл, Шаблон и т. д.) исключены. Просто отбросить все URL с двоеточием нельзя: заголовки новостей часто содержат двоеточие («Александр Егоров: Нам не пришлось что-то выдумывать»), поэтому сравнивается именно префикс до двоеточия со списком пространств имён из конфига.

- **robots.txt** загружается один раз при старте (нашим `User-Agent`) и проверяется при добавлении URL в очередь.

- **Канонические URL.** Если страница открылась по алиасу или перенаправлению, документ сохраняется под URL из `<link rel="canonical">`, а этот канонический URL помечается в очереди обработанным, чтобы страница не загружалась дважды.

- **Порядок обхода** — FIFO по полю `added`; URL, на которых были сбои, уходят в конец очереди (сортировка по `tries`).

6 Обработка ошибок HTTP

Ни одна ошибка одной страницы не прерывает обкачку:

Ситуация	Реакция
200, `text/html`	документ сохраняется
304 Not Modified	документ не изменился, обновляется только `crawl_timestamp`
404, 410 и другие 4xx	URL помечается `failed` и больше не запрашивается
200, но не HTML	`failed`
429, 5xx	библиотека `urllib3` повторяет запрос `retry_attempts` раз с нарастающей паузой (1, 2, 4 с), заголовок `Retry-After` учитывается
таймаут, обрыв соединения, 429/5xx после всех повторов	`tries` увеличивается, URL уходит в конец очереди; после `retry_attempts` неудач — состояние `error`, при следующем запуске такие URL возвращаются в очередь

7 Возобновление работы

Контрольная точка — сама очередь. URL получает состояние `done` только после того, как документ записан в базу. Поэтому робот можно остановить в любой момент: `Ctrl+C` или `SIGTERM` (робот дозагружает текущую страницу и выходит; повторный сигнал завершает его сразу), а при аварийном завершении (даже `kill -9`) теряется максимум одна страница — она останется в состоянии `new` и будет загружена заново. При повторном запуске робот берёт из очереди необработанные URL и продолжает с того места, где остановился; стартовые URL добавляются идемпотентно.

8 Переобкачка изменившихся документов

При каждом запуске URL в состоянии `done`, обкачанные более `refresh_interval_days` дней назад, возвращаются в очередь с признаком `refresh`. Запуск робота по расписанию (например, `cron`) даёт периодическую переобкачку. Для таких страниц отправляется условный запрос `If-Modified-Since` (значение берётся из сохранённого `Last-Modified`):

- 304 — обновляется только `crawl_timestamp`;
- 200 — сравнивается MD5-хэш *текста статьи*. Хэш считается не от всего HTML, поскольку в странице MediaWiki есть служебные данные (отчёт парсера, время генерации), которые могут меняться без изменения статьи. Совпал — обновляется только дата; не совпал — документ перезаписывается, а прежние хэш и дата добавляются в `history`.

Когда число документов достигло `max_documents`, робот берёт из очереди только URL на переобкачку и новые страницы не загружает.

9 Проверка

Файл `test_main.py` содержит пять автоматических проверок на подставной HTTP-сессии и MongoDB в памяти (`mongomock`): обход с дедупликацией и фильтрацией ссылок (категории, внешние домены, query, заголовок с двоеточием), перенаправление в закрытое пространство имён (документ не сохраняется); остановка посреди обкачки с последующим продолжением (каждый URL загружается ровно один раз), переобкачка (изменившийся документ получает запись в `history`, неизменившийся — нет), ошибки HTTP (404, 503, обрыв соединения). Кроме того, выполнен контрольный прогон по живому сайту `ru.wikinews.org` в три этапа: первичная обкачка до 100 документов; «перезапуск» с продолжением из очереди до 200 документов; переобкачка всех документов, у которых срок давности принудительно считался истёкшим.

10 Статистика контрольного прогона

Значения получены командой `python3 main.py config.yaml --stats` и по логу прогона; задержка между запросами 1 с.

Показатель	Значение
Количество документов	200
Размер «сырых» данных	25,1 МБ
Размер выделенного текста	1,5 МБ
Средний размер документа	125,7 КБ
Средний объём текста в документе	7,5 КБ
Скорость обкачки	100 док. за 119 с и 100 док. за 139 с (≈0,78 док/с, ≈1,3 с на документ)
Ошибки HTTP и сбои соединения	0
URL в очереди после 200 документов	222 обработано, 3497 ожидают
Из них адреса-алиасы (канонический URL другой)	22 из 222
Повторная загрузка при перезапуске	0 (каждый URL запрошен один раз)
Переобкачка 200 документов	178 ответов 304, 22 ответа 200 с тем же хэшем, изменившихся 0

Очередь после 200 документов насчитывает уже около 3700 адресов: обход по ссылкам из тела статьи быстро выходит за пределы стартовой страницы, так что лимит в 30000 документов достижим. Страницы-алиасы — в основном ссылки на перенаправления: например, заголовок с запятой и без неё ведёт на один и тот же документ; в таких случаях сохраняется ровно один документ под каноническим URL. Если канонический URL указывает в закрытое пространство имён (например, перенаправление в категорию), документ не сохраняется.

Для всего корпуса статистика получается той же командой по базе после полной обкачки:

```
python3 main.py config.yaml --stats
```

2 Исходный код

Программа `main.py` состоит из класса `Crawler` и вспомогательных функций. Основные части: `normalize` (нормализация URL), `wanted` (фильтр ссылок), `enqueue`, `requeue`, `next_item` (очередь), `crawl_one` (загрузка и сохранение одной страницы), `finish` (учёт результата) и `run` (основной цикл).

Нормализация URL (`SAFE` — символы, которые MediaWiki не кодирует в заголовках) и фильтр ссылок (домен, путь, пространство имён, robots.txt):

```
def normalize(url, base=None):
    p = urlparse(urljoin(base, url) if base else url)
    path = unquote(p.path).replace(' ', '_')
    return 'https://' + p.netloc.lower() + quote(path, safe=SAFE)

def wanted(self, url):
    p = urlparse(url)
    if p.netloc != self.domain or not p.path.startswith('/wiki/'):
        return False
    title = unquote(p.path[len('/wiki/'):])
    ns, colon, rest = title.partition(':')
    return (not (colon and ns in self.skip)
            and self.robots.can_fetch(self.ua, url))
```

Очередь в MongoDB: добавление без повторов, выбор следующего URL, возврат в очередь устаревших документов:

```
def enqueue(self, urls):
    now = time.time()
    ops = [UpdateOne({"_id": u}, {"$setOnInsert": {"state": "new",
           "tries": 0, "refresh": False, "added": now}}, upsert=True)
           for u in set(urls) if self.wanted(u)]
    if ops:
        self.queue.bulk_write(ops, ordered=False)

def next_item(self):
    flt = {"state": "new"}
    if self.limit and self.docs.estimated_document_count() >= self.limit:
        flt["refresh"] = True # corpus is full: only refresh
    return self.queue.find_one(
        flt, sort=[("tries", ASCENDING), ("added", ASCENDING)])

def requeue(self):
    cutoff = int(time.time() - self.refresh_days * 86400)
    self.queue.update_many({"state": "done", "done_at": {"$lt": cutoff}},
        {"$set": {"state": "new", "tries": 0, "refresh": True}})
    self.queue.update_many({"state": "error"},
        {"$set": {"state": "new", "tries": 0}})
```

Загрузка одной страницы: условный запрос, разбор ошибок HTTP, хэш текста статьи, запись документа с историей версий:

```
def crawl_one(self, url):
    old = self.docs.find_one({"url": url}, {"html_content": 0})
    headers = ({"If-Modified-Since": old["last_modified"]}
               if old and old.get("last_modified") else {})
    try:
        r = self.session.get(url, headers=headers, timeout=self.timeout)
    except requests.RequestException:
        return "error"
    if r.status_code == 304: # not modified
        self.docs.update_one({"url": url},
            {"$set": {"crawl_timestamp": int(time.time())}})
        return "done"
    if r.status_code == 429 or r.status_code >= 500:
        return "error" # retry later
    if r.status_code != 200 or "text/html" not in r.headers.get(
            "Content-Type", ""):
        return "failed" # never retry
    soup = BeautifulSoup(r.text, "html.parser")
    body = soup.select_one("#mw-content-text") or soup
    text = body.get_text(" ", strip=True)
    h = hashlib.md5(text.encode()).hexdigest()
    ... # canonical URL, see src
    if old and old["content_hash"] == h: # same text: only the date
        self.docs.update_one({"url": doc_url},
            {"$set": {"crawl_timestamp": now}})
    else:
        update = {"$set": {"url": doc_url, "html_content": r.text,
                  "source": self.name, "crawl_timestamp": now,
                  "content_hash": h, ...}}
        if old: # changed: keep history
            update["$push"] = {"history": {
                "content_hash": old["content_hash"],
                "crawl_timestamp": old["crawl_timestamp"]}}
        self.docs.update_one({"url": doc_url}, update, upsert=True)
    self.enqueue(normalize(a["href"], url) for a in body.select("a[href]"))
    return "done"
```

Основной цикл и остановка по сигналу (очередь в базе — контрольная точка):

```
def run(self):
    self.requeue()
    self.enqueue(normalize(s) for s in self.seeds)
    while not self.stop.is_set():
        item = self.next_item()
        if not item:
            break
        self.finish(item, self.crawl_one(item["_id"]))
        self.stop.wait(self.delay) # returns at once on Ctrl+C
```

3 Выводы

Выполнив вторую лабораторную работу по курсу «Информационный поиск», я написал поискового робота для русских Викиновостей. Основное, что я узнал нового:

- устройство краулера на практике: очередь URL, нормализация адресов, обход в ширину, вежливые задержки, `robots.txt`, идентифицирующий `User-Agent`;
- очередь удобно хранить в самой базе данных: тогда она одновременно защищает от повторных загрузок (`upsert` по URL) и служит контрольной точкой — остановить робота можно в любой момент, не заводя отдельного файла состояния;
- ошибки HTTP нужно делить на постоянные (404, не-HTML) и временные (429, 5xx, таймауты): первые не повторяются, вторые откладываются в конец очереди с ограничением числа попыток, и ни одна не должна останавливать всю обкачку;
- переобкачка по изменению: условный запрос `If-Modified-Since` экономит трафик, а сравнение хэша текста статьи позволяет не переписывать неизменившиеся документы и хранить историю версий изменившихся;
- реальность внесла поправки: заголовки новостей содержат двоеточие, поэтому служебные страницы (`Категория:`, `Служебная:`, ...) нельзя отсеивать по одному символу — понадобился список пространств имён; одна и та же страница доступна по нескольким адресам, поэтому документы хранятся под каноническим URL (в контрольном прогоне такими оказались 22 из 222 адресов);
- «сырой» HTML страницы Викиновостей в 17 раз больше полезного текста (125,7 КБ против 7,5 КБ на документ), что нужно учитывать при выборе хранилища и при построении индекса.

Викиновости открыты для автоматического доступа при соблюдении правил (идентифицирующий `User-Agent`, `robots.txt`, пауза между запросами), поэтому для робота выбран именно этот источник. Скорость обкачки упирается в паузу между запросами (1 с): корпус из десятков тысяч документов набирается за часы. Собранные документы с датами обкачки и историей версий будут использоваться в следующих лабораторных работах для построения индекса.

Список литературы

[1] Маннинг, Рагхаван, Шютце *Введение в информационный поиск* — Издательский дом «Вильямс», 2011. Перевод с английского: доктор физ.-мат. наук Д. А. Клюшина — 528 с. (ISBN 978-5-8459-1623-4 (рус.))

[2] Список использованных источников оформлять нужно по ГОСТ Р 7.05-2008

[3] Викиновости, свободный источник новостей // https://ru.wikinews.org

[4] MongoDB Documentation // https://www.mongodb.com/docs/

[5] PyMongo Documentation // https://pymongo.readthedocs.io

[6] PyYAML Documentation // https://pyyaml.org/wiki/PyYAMLDocumentation

[7] A Standard for Robot Exclusion // https://www.robotstxt.org/orig.html

[8] Wikimedia Foundation User-Agent Policy // foundation.wikimedia.org

[9] Requests: HTTP for Humans // https://docs.python-requests.org

[10] Beautiful Soup Documentation // https://www.crummy.com/software/BeautifulSoup/bs4/doc/